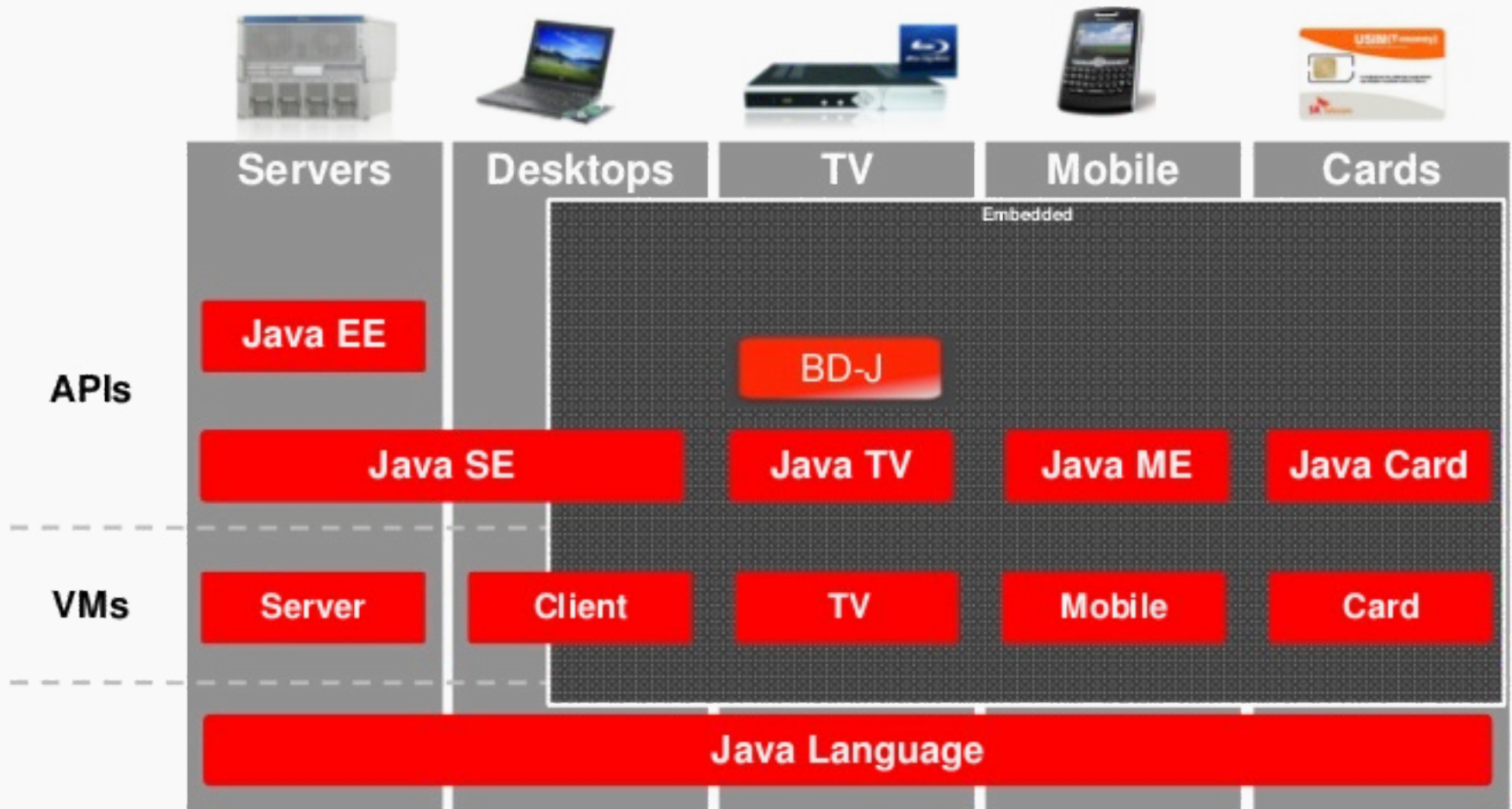


14. Java EE Full Profile

Java Platforms



Основные концепции

- Приложения строятся из компонентов, работающих под управлением контейнеров.
- Используются следующие принципы:
 - Inversion of Control (IoC) + Contexts & Dependency Injection (CDI).
 - Location Transparency.

Принципы IoC и CDI

- IoC (применительно к Java EE):
 - Жизненным циклом компонента управляет контейнер (а не программист).
 - За взаимодействие между компонентами отвечает тоже контейнер.
- CDI — позволяет снизить (или совсем убрать) зависимость компонента от контейнера:
 - Не требуется реализации каких-либо интерфейсов.
 - Не нужны прямые вызовы API.
 - Реализуется через аннотации.

Пример CDI (JSF Managed Bean)

```
@ManagedBean(name="message")
@SessionScoped
public class MessageBean implements Serializable {
    //business logic and whatever methods
}
```

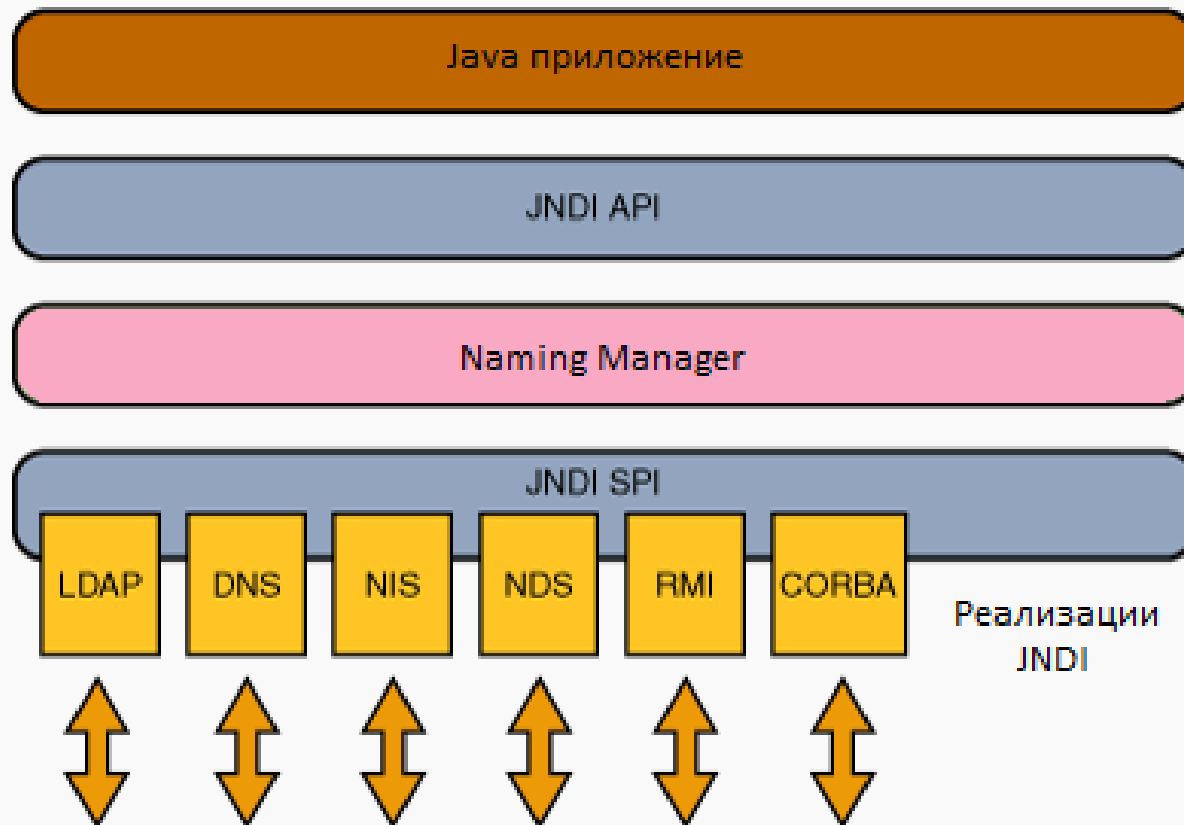
```
@ManagedBean
@SessionScoped
public class HelloBean implements Serializable {

    @ManagedProperty(value="#{message}")
    private MessageBean messageBean;

    //must provide the setter method
    public void setMessageBean(MessageBean messageBean) {
        this.messageBean = messageBean;
    }
}
```

Java Naming & Directory Interface (JNDI)

JNDI — это набор Java API, организованный в виде службы каталогов, который позволяет Java-клиентам открывать и просматривать данные и объекты по их именам (C) Wikipedia.



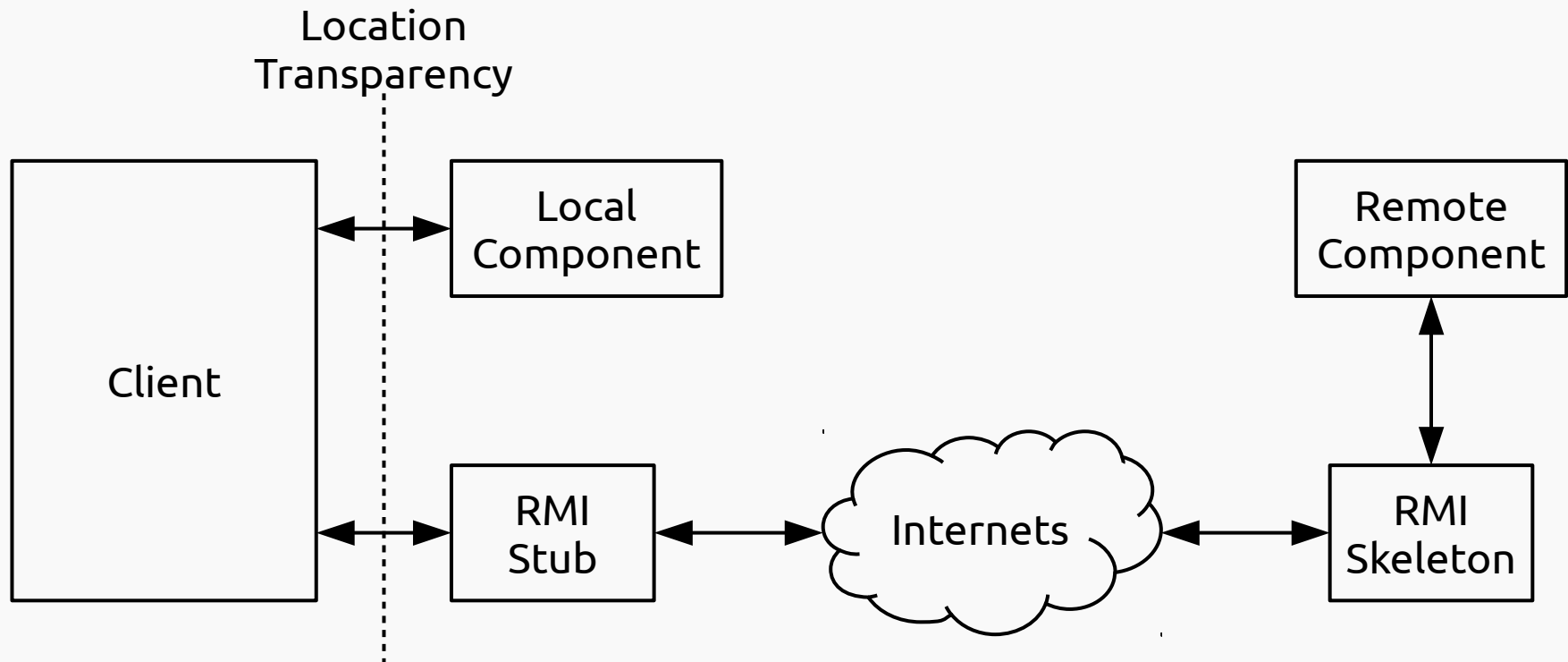
Два варианта использования JNDI:

- CDI (аннотации) — работает только в managed компонентах.
- Прямой вызов API — работает везде.

```
// Пример получения ссылки на JDBC datasource.  
DataSource dataSource = null;  
try {  
    // Инициализируем контекст по умолчанию.  
    Context context = new InitialContext();  
    dataSource =  
        (DataSource) context.lookup("Database");  
} catch (NamingException e) {  
    // Ссылка не найдена  
}
```

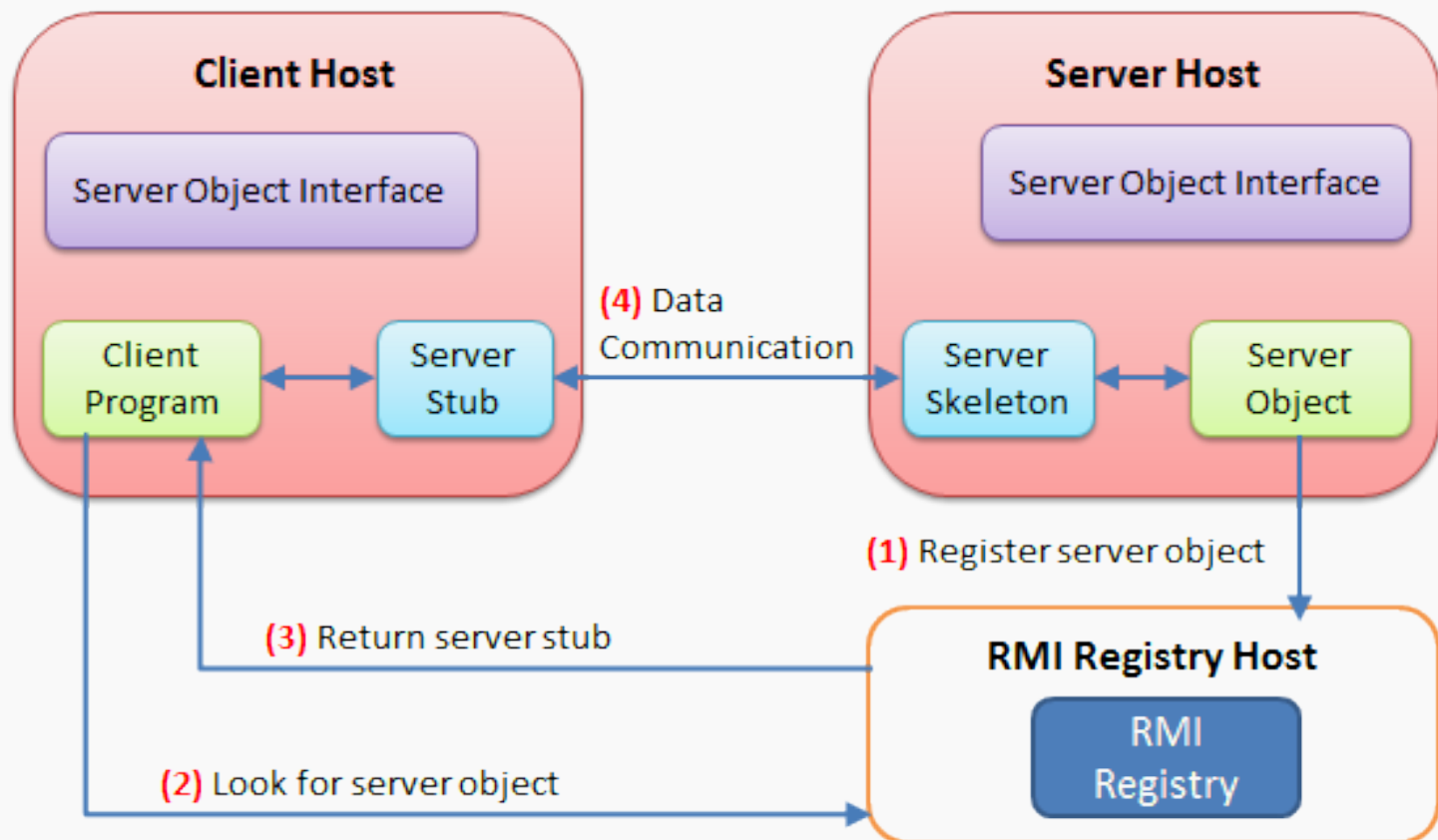
Принцип Location Transparency

Благодаря CDI не важно, где физически расположен вызываемый компонент — за его вызов отвечает контейнер.



Remote Method Invocation

RMI — Java API, позволяющий вызывать методы удалённых объектов.



- В общем случае, объекты передаются по значению (копии).
- Передаваемые объекты должны быть Serializable.

Пример реализации RMI — сервер

```
public class PrimeNumbersSearchServer implements ClientRegister {  
  
    ...  
  
    public static void main(String[] args) {  
        PrimeNumbersSearchServer server =  
            new PrimeNumbersSearchServer();  
        try {  
            ClientRegister stub = (ClientRegister)  
                UnicastRemoteObject.exportObject(server, 0);  
            Registry registry = LocateRegistry.createRegistry(12345);  
            registry.bind("ClientRegister", stub);  
            server.startSearch();  
        } catch (Exception e) {  
            System.out.println ("Error occurred: " + e.getMessage());  
            System.exit (1);  
        }  
    }  
}
```

Пример реализации RMI — клиент

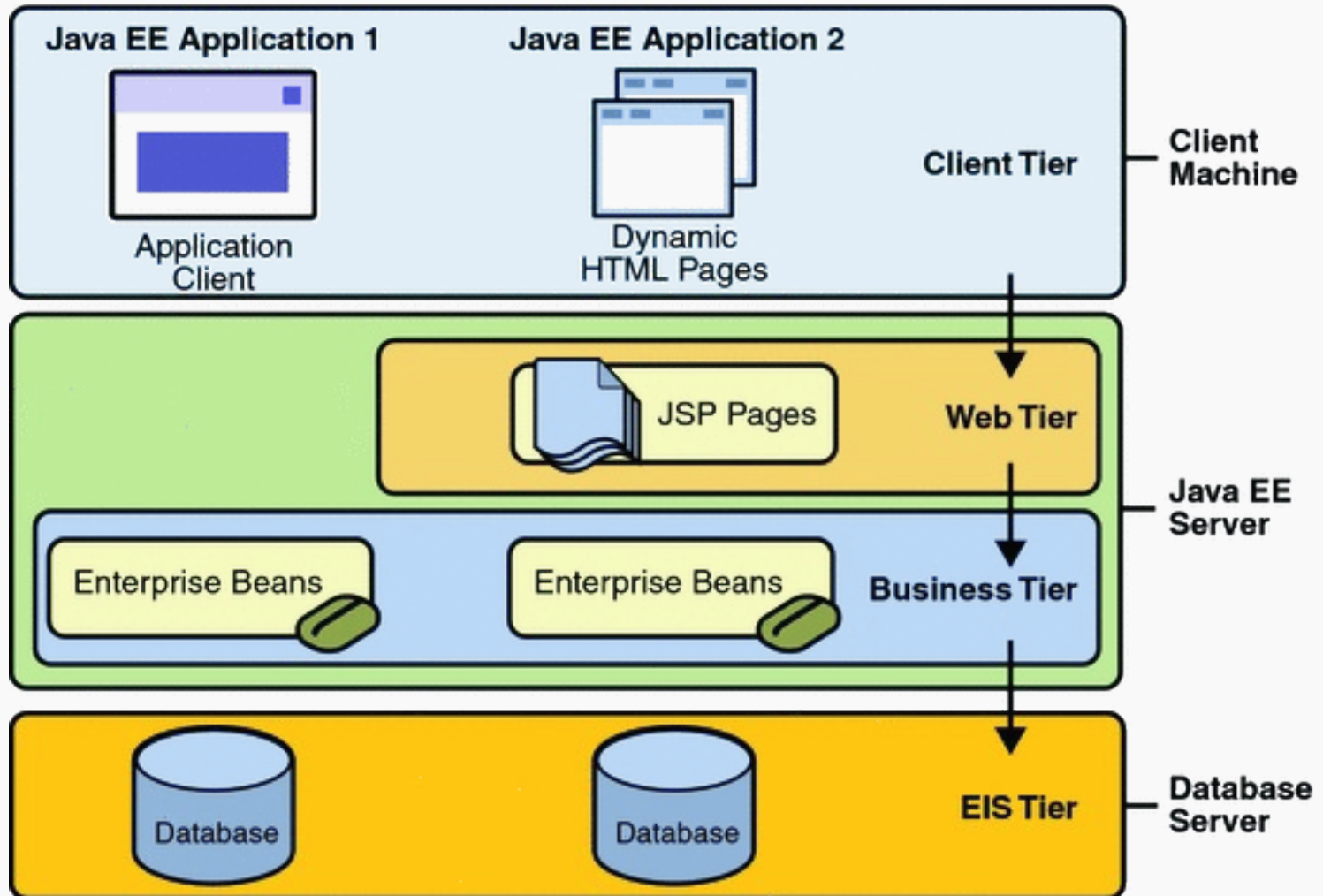
```
public class PrimeNumbersSearchClient implements PrimeChecker {

    ...

    public static void main(String[] args) {
        PrimeNumbersSearchClient client =
            new PrimeNumbersSearchClient();

        try {
            Registry registry = LocateRegistry.getRegistry(null, 12345);
            ClientRegister server =
                (ClientRegister) registry.lookup("ClientRegister");
            PrimeChecker stub = (PrimeChecker)
                UnicastRemoteObject.exportObject(client, 0);
            server.register(stub);
        } catch (Exception e) {
            System.out.println ("Error occurred: " + e.getMessage());
            System.exit (1);
        }
    }
}
```

Java EE Application Tiers

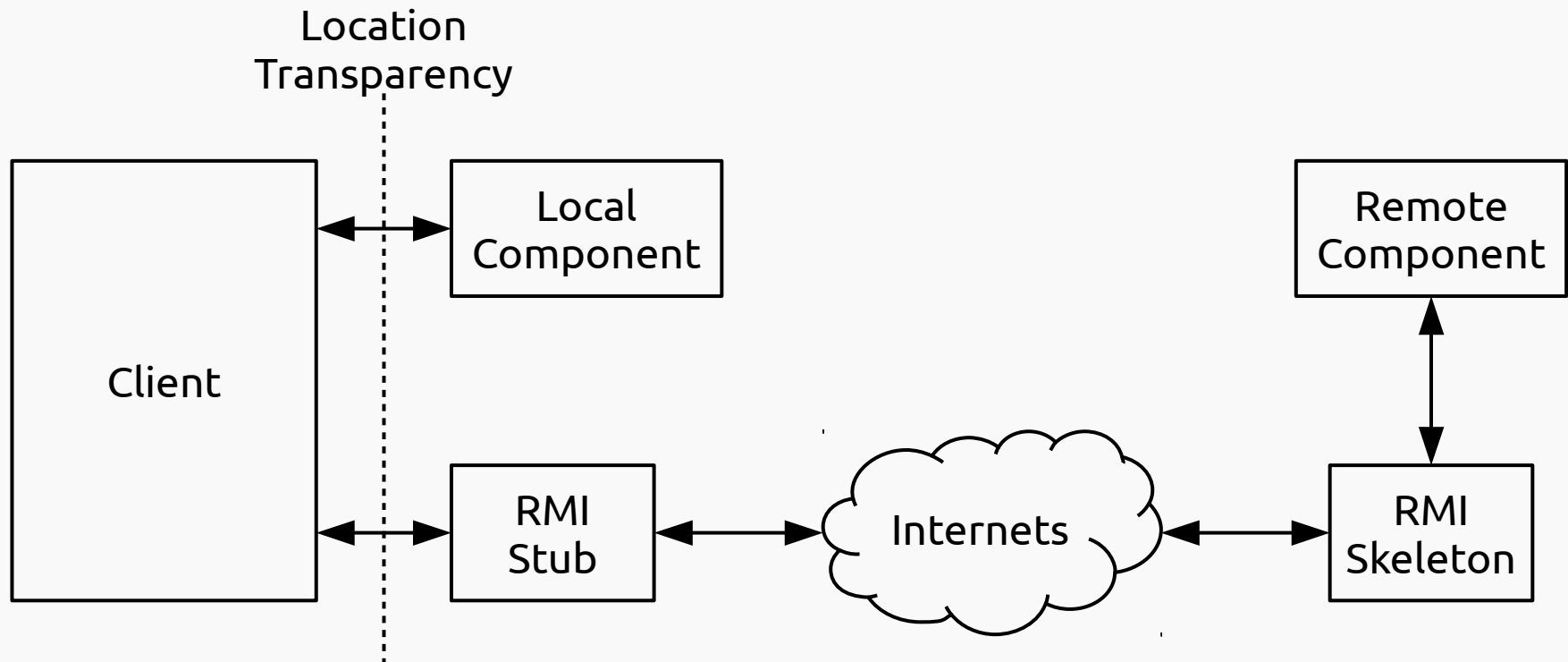


Профили платформы Java EE

- Появились в Java EE 6.
- Позволяют сделать более «лёгкими» приложения, которым не нужен полный стек технологий Java EE.
- Существует только 2 профиля — Full и Web.
- Сервер приложений может реализовывать спецификации не всей платформы, а конкретного профиля.

Принцип Location Transparency

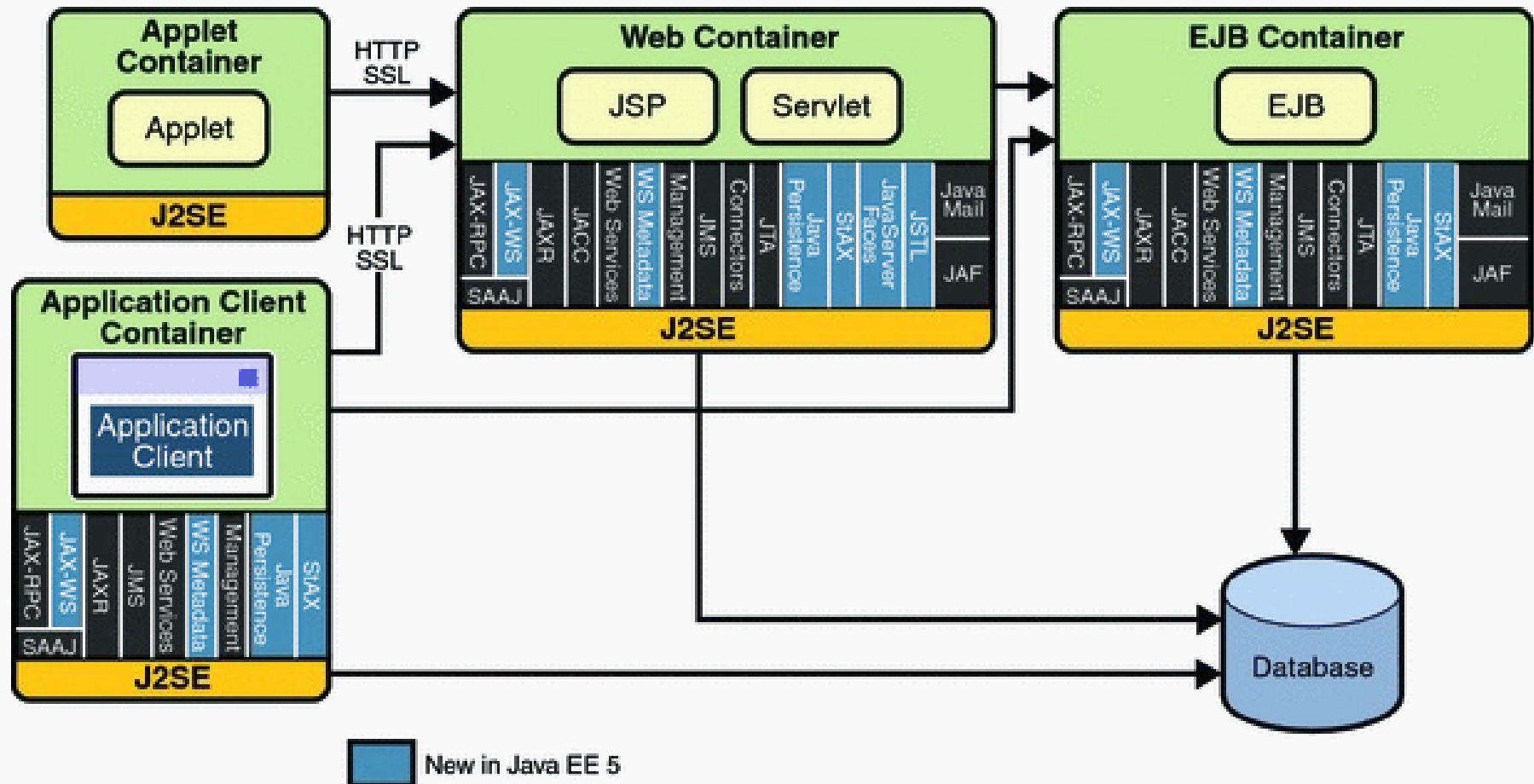
Благодаря CDI не важно, где физически расположен вызываемый компонент — за его вызов отвечает контейнер.



Java EE Full & Web Profiles

API	Web Profile	Full Profile		API	Web Profile	Full Profile
Servlet	+	+		JTA	+	+
JSP	+	+		JMS		+
JSTL	+	+		JavaMail		+
EL	+	+		JAX-WS		+
JSF	+	+		JAX-RS		+
CDI	+	+		JAXB		+
EJB Lite	+	+		JACC		+
EJB Full		+		JCA		+
JPA	+	+				

Архитектура приложения Java EE Full Profile



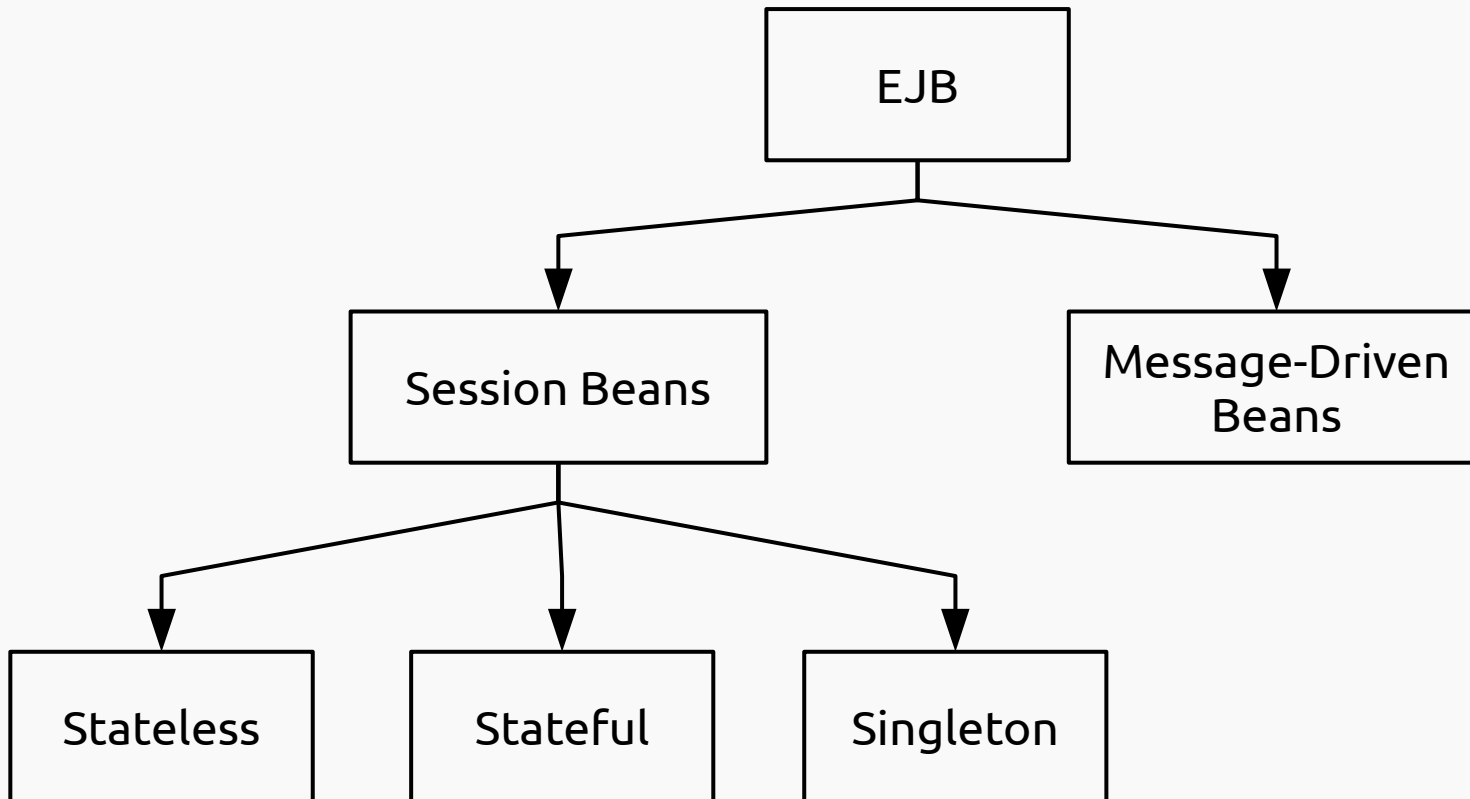
15. Enterprise Java Beans

EJB — технология разработки серверных компонентов, реализующих бизнес-логику.

Особенности EJB:

- Возможность локального и удалённого доступа.
- Возможность доступа через JNDI или Dependency Injection.
- Поддержка распределённых транзакций (с помощью JTA).
- Поддержка событий.
- Жизненным циклом управляет EJB-контейнер (в составе сервера приложений).

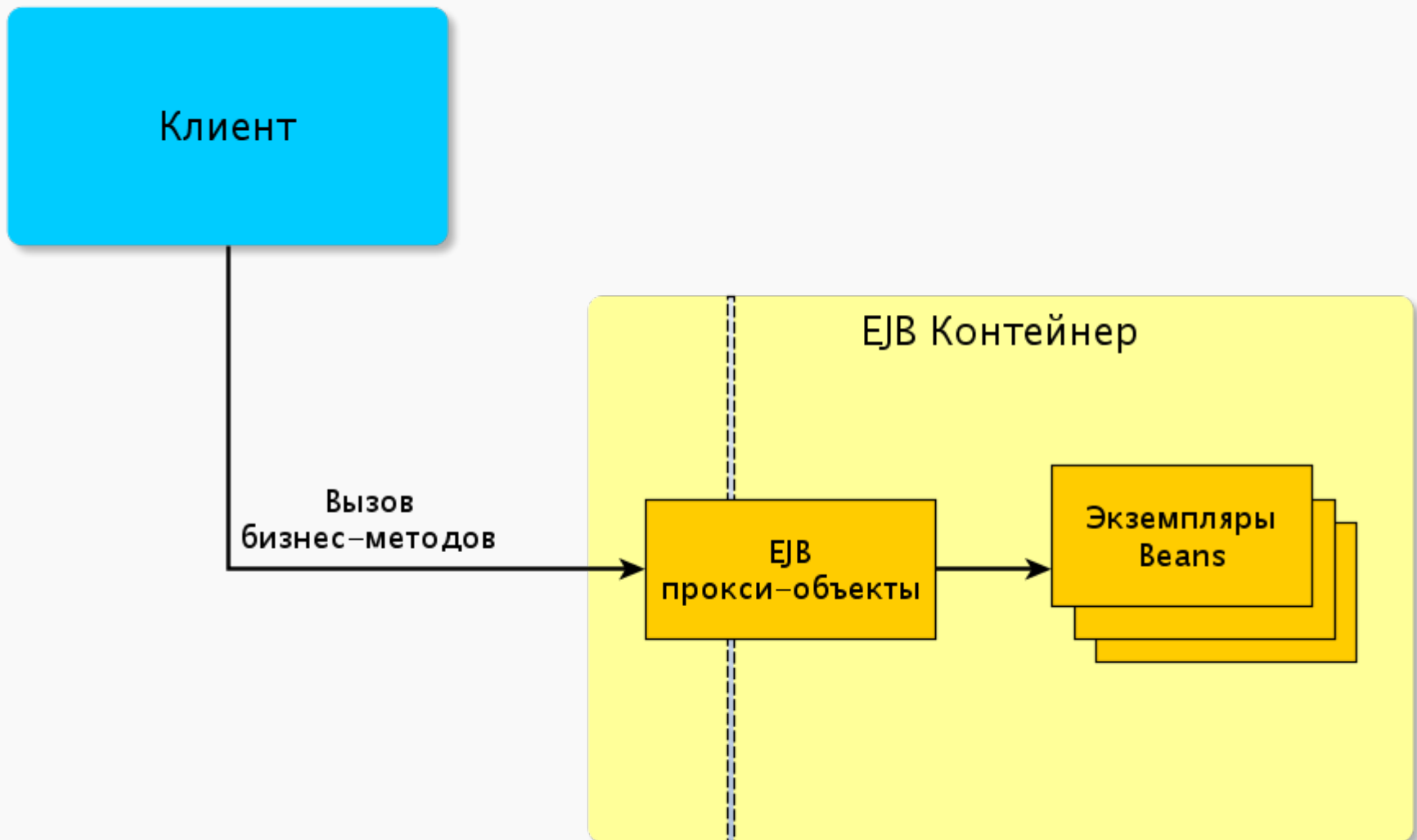
Виды ЕJB



Компонентная модель EJB

- EJB-компоненты инкапсулируются контейнером.
- Контейнер предоставляет клиентам прокси-объекты (proxy objects) для доступа к EJB-компонентам.
- Прокси-объекты реализуют бизнес-интерфейсы (Business Interfaces).
- Клиенты вызывают методы бизнес-интерфейсов.

Компонентная модель EJB (продолжение)



Варианты доступа к EJB

- Локальный (Local) — из той же самой JVM.
- Удалённый (Remote) — из другой JVM.
- Через веб-сервис.

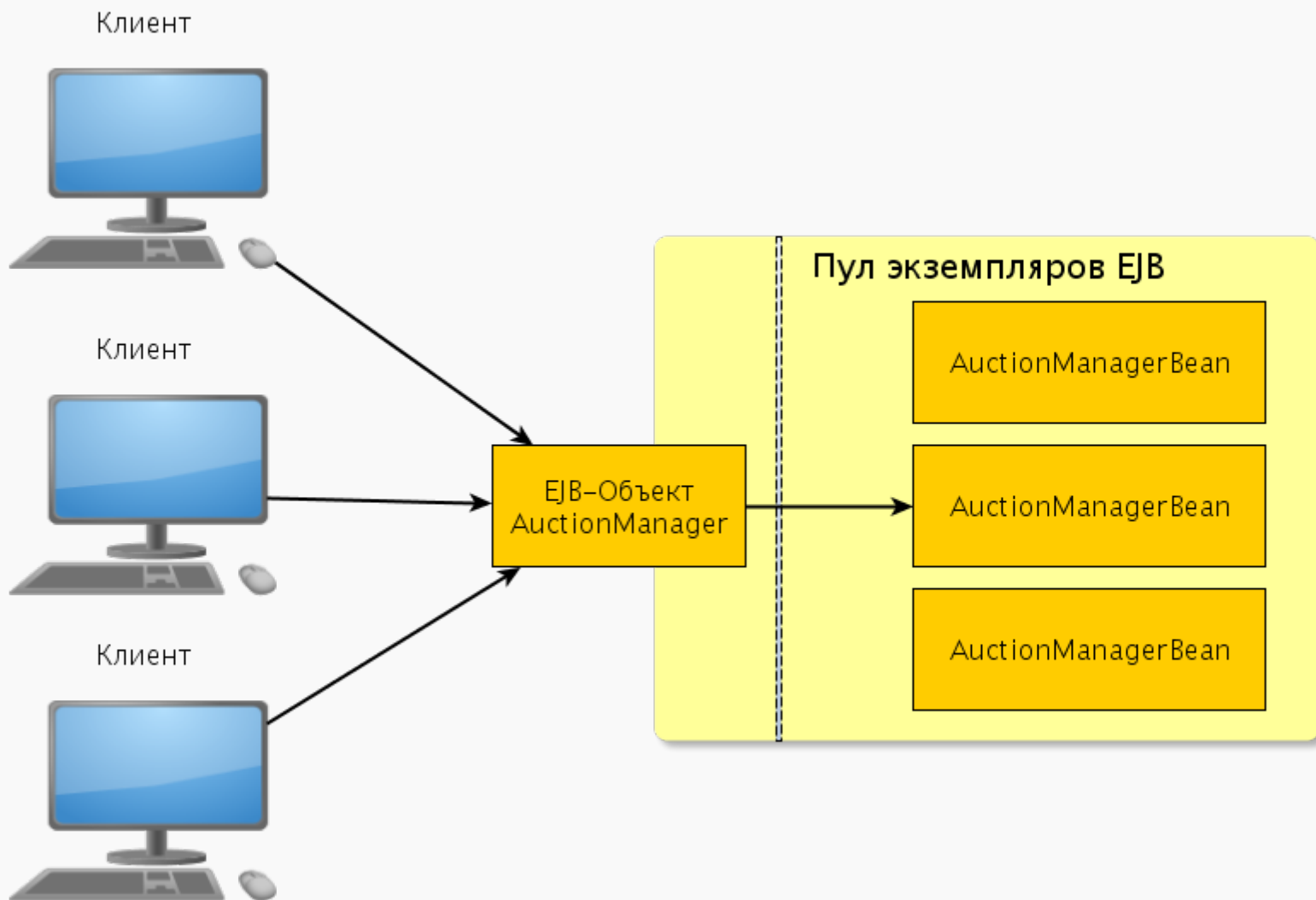
Session Beans

- Предназначены для синхронной обработки вызовов.
- Вызываются посредством обращения через API.
- Могут вызываться локально или удалённо.
- Могут быть endpoint'ами для веб-сервисов.
- CDI — аннотация @EJB.
- Не обладают свойством персистентности.
- Можно формировать пулы бинов (за исключением @Singleton).

Stateless Session Beans

- Не сохраняют состояние между последовательными обращениями клиента.
- Нет привязки к конкретному клиенту.
- Хорошо масштабируются.
- Объявление — аннотация `@Stateless`.

Stateless Session Beans (продолжение)



Stateless Session Beans (пример)

```
package converter.ejb;

import java.math.BigDecimal;
import javax.ejb.*;

@Stateless
public class ConverterBean {
    private BigDecimal yenRate = new BigDecimal("83.0602");
    private BigDecimal euroRate = new BigDecimal("0.0093016");

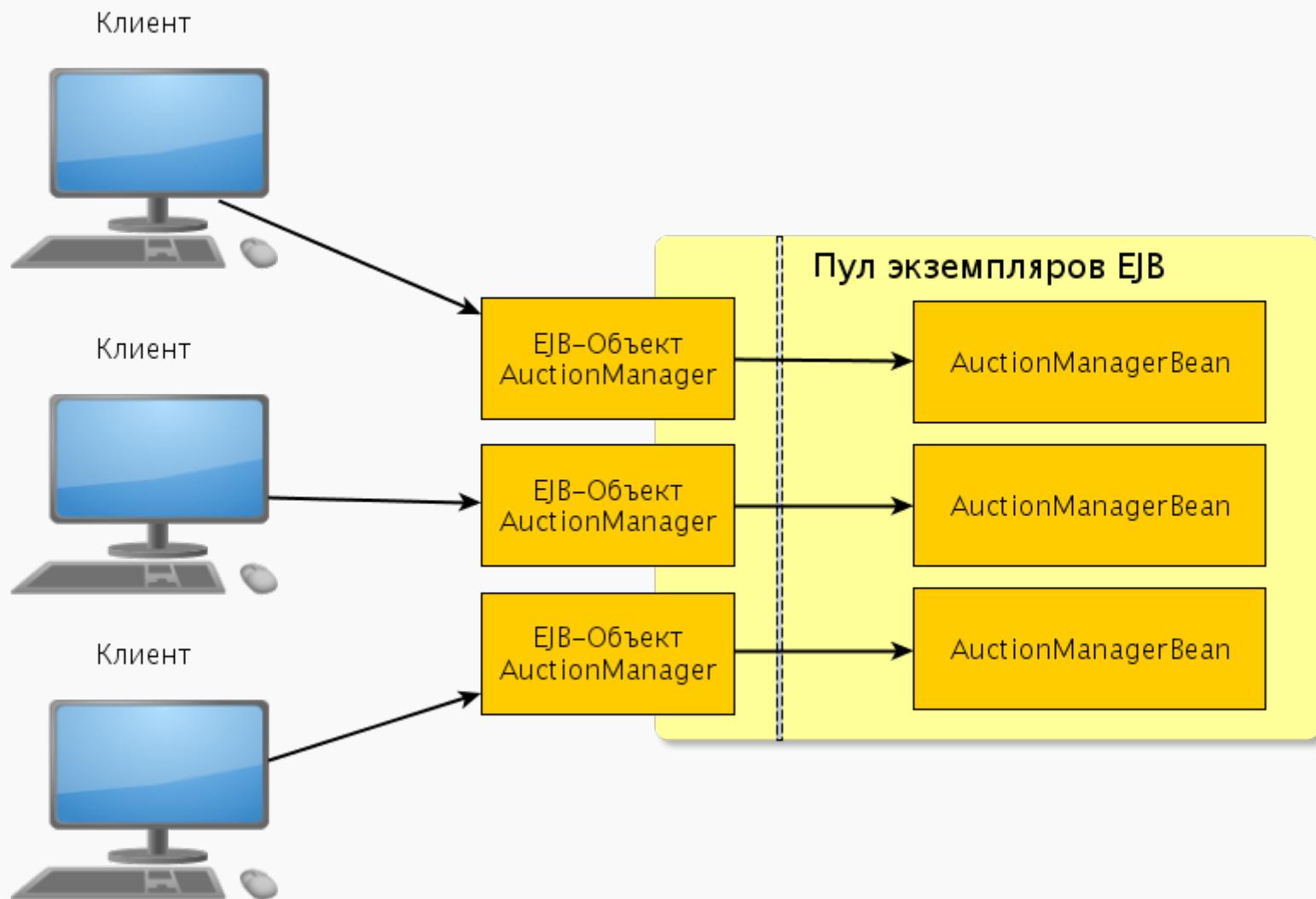
    public BigDecimal dollarToYen(BigDecimal dollars) {
        BigDecimal result = dollars.multiply(yenRate);
        return result.setScale(2, BigDecimal.ROUND_UP);
    }

    public BigDecimal yenToEuro(BigDecimal yen) {
        BigDecimal result = yen.multiply(euroRate);
        return result.setScale(2, BigDecimal.ROUND_UP);
    }
}
```

Stateful Session Beans

- «Привязываются» к конкретному клиенту.
- Можно сохранять контекст в полях класса.
- Масштабируются хуже, чем `@Stateless`.
- Объявление — аннотация `@Stateful`.

Stateful Session Beans (продолжение)



Stateful Session Beans (пример)

```
@Stateful
@StatefulTimeout(unit = TimeUnit.MINUTES, value = 20)
public class CartBean implements Cart {

    @PersistenceContext(unitName = "pu", type = PersistenceContextType.EXTENDED)
    private EntityManager entityManager;

    private List products;

    @PostConstruct
    private void initializeBean(){
        products = new ArrayList<>();
    }

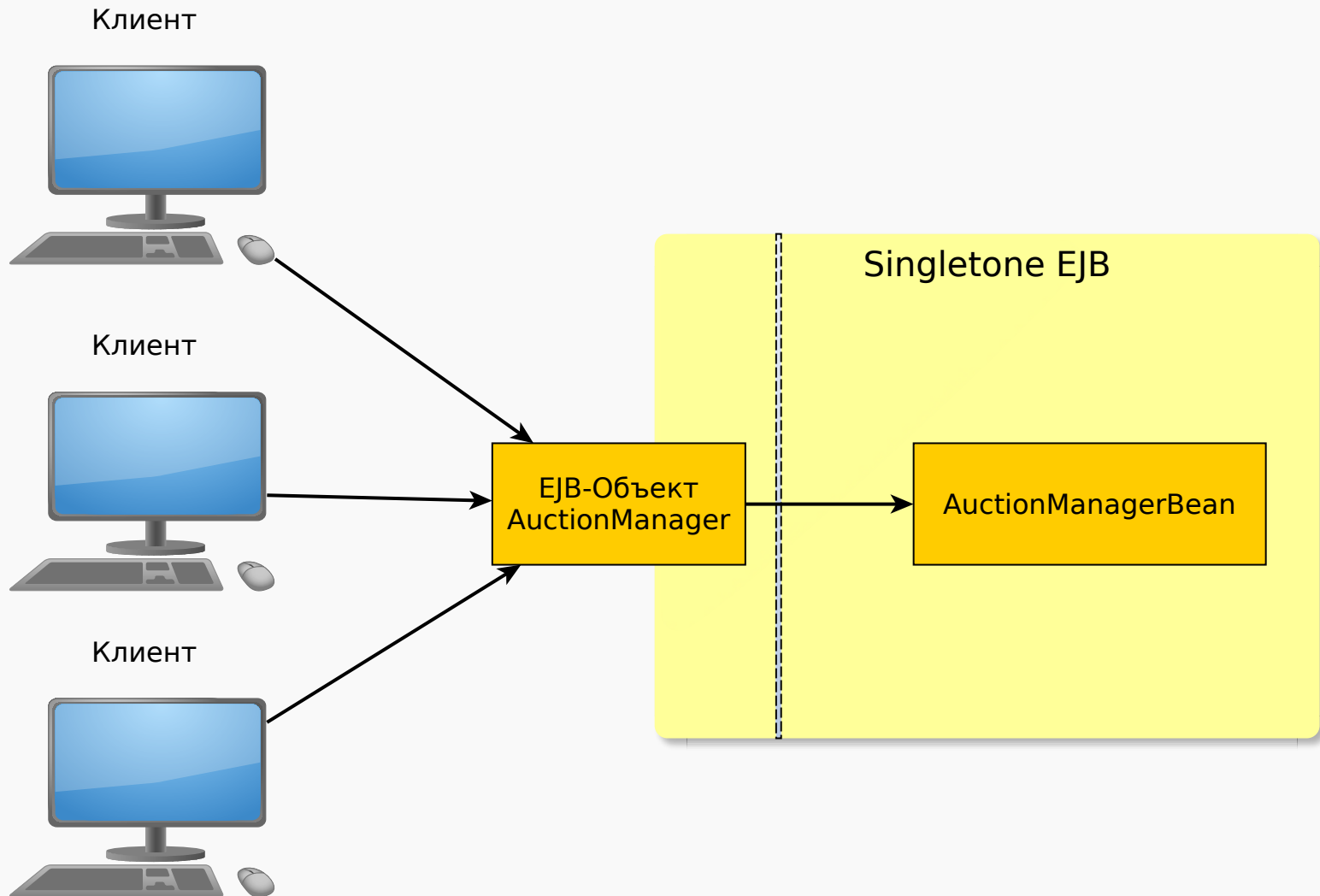
    @Override
    public void addProductToCart(Product product) {
        products.add(product);
    }

    @Override
    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void checkOut() {
        for(Product product : products){
            entityManager.persist(product);
        }
        products.clear();
    }
}
```

Singleton Session Beans

- Контейнер гарантирует существование строго одного экземпляра такого бина.
- Объявление — аннотация `@Singleton`.

Singleton Session Beans (продолжение)



- 1) Создание бизнес-интерфейса компонента (Business Interface).
- 2) Создание класса компонента, реализующего бизнес-интерфейс.
- 3) Конфигурация компонента с помощью аннотаций и/или дескриптора развёртывания.

Бизнес-интерфейс содержит заголовки всех методов, реализующих бизнес-логику компонента.

Может быть локальным (`@Local`) и удалённым (`@Remote`).

Пример:

```
import javax.ejb.*;

@Remote
public interface Hello {
    public String sayHello();
}
```

Создание класса компонента

Компонент должен реализовывать все методы бизнес-интерфейса.

Может содержать методы, реагирующие на события жизненного цикла компонента.

Пример:

```
import javax.ejb.*;

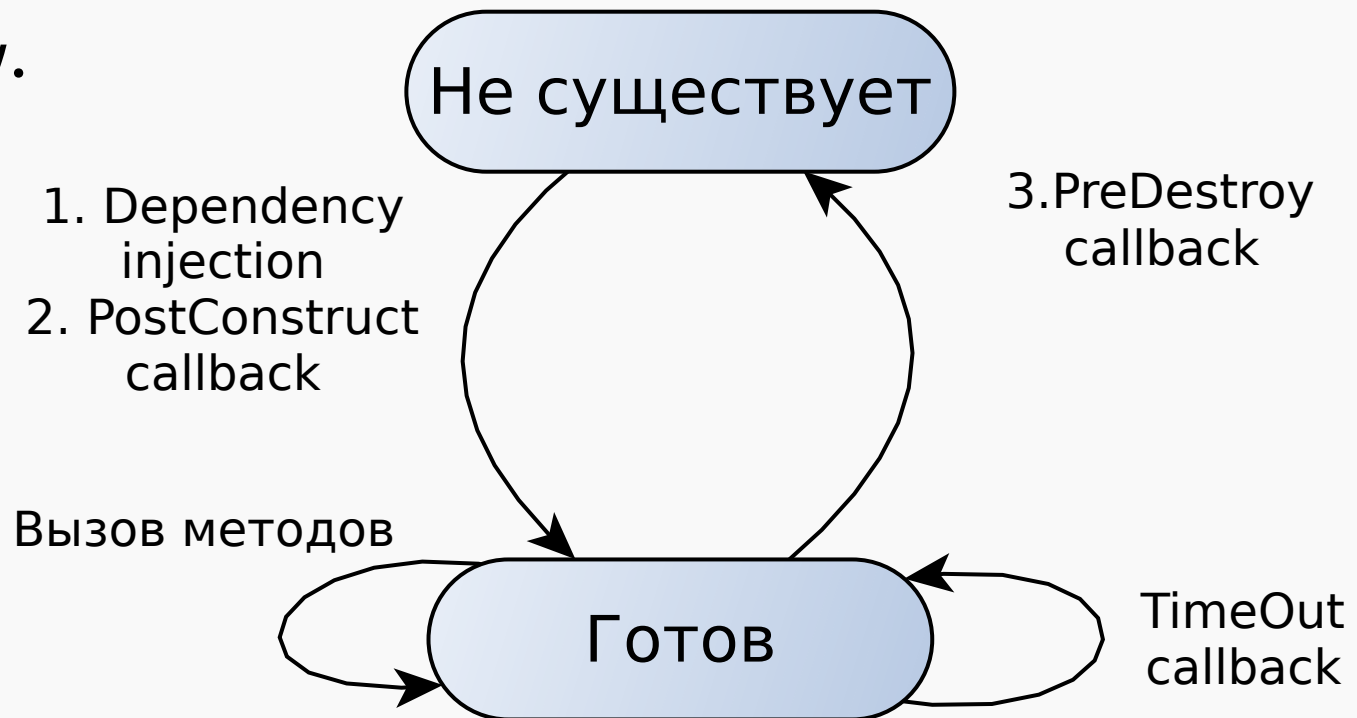
@Stateless
public class HelloBean implements Hello {
    public String sayHello() {
        return "Hello World!";
    }
}
```

Требования к классам Session Beans

- Должен быть `public` классом верхнего уровня.
- Не должен быть `final` и/или `abstract`.
- Должен иметь `public` конструктор без параметров.
- Не должен переопределять метод `finalize()`.
- Должен реализовывать все методы бизнес-интерфейса(-ов).

Обработчики событий жизненного цикла:

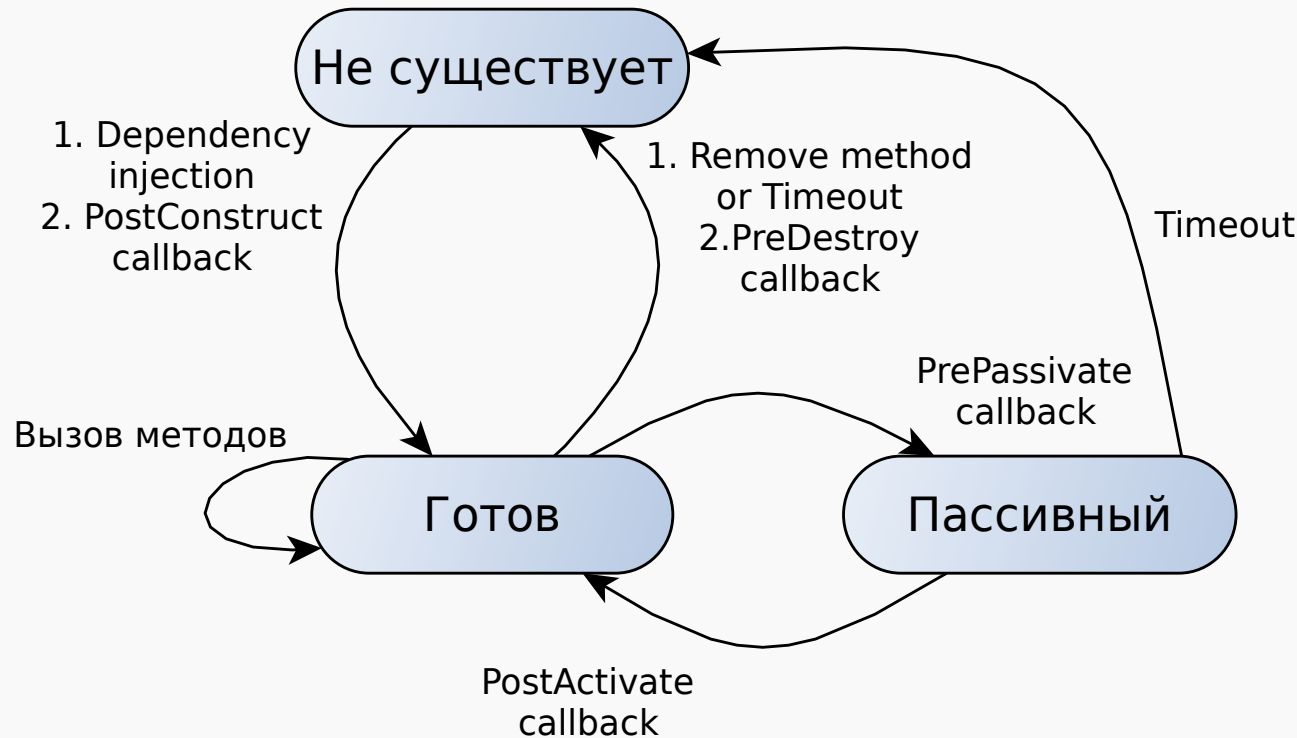
- @PostConstruct;
- @PreDestroy.



Жизненный цикл Stateful Session Beans

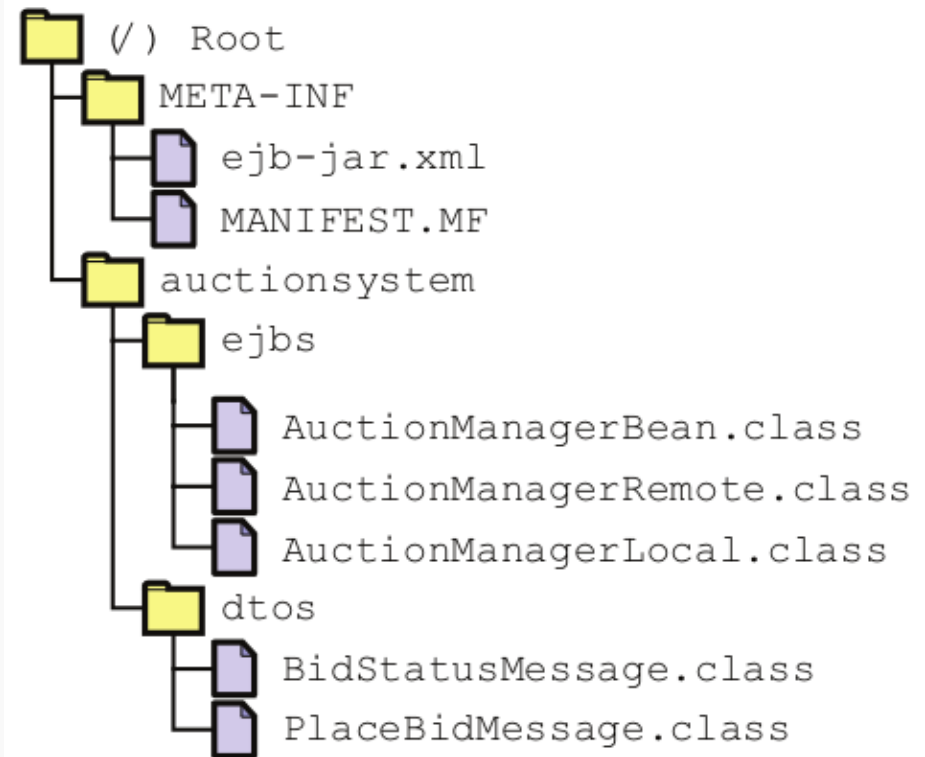
Обработчики событий жизненного цикла:

- `@PostConstruct`;
- `@PreDestroy`;
- `@PostActivate`;
- `@PrePassivate`.



Развёртывание Session Beans на сервере приложений

- 1) Создание дескриптора развёртывания (опционально).
- 2) Создание jar-архива с компонентами.
- 3) Развёртывание архива на сервере приложений.



Обращение к Session Beans

С помощью Dependency Injection:

```
import javax.ejb.*;

public class HelloClient {

    @EJB
    private static Hello hello;

    public static void main (String args[]) {
        System.out.println(hello.sayHello())
    }

}
```

Java Transaction API (JTA)

Интерфейс для управления транзакциями в EJB-компонентах. Позволяет открывать, закрывать и откатывать транзакции.

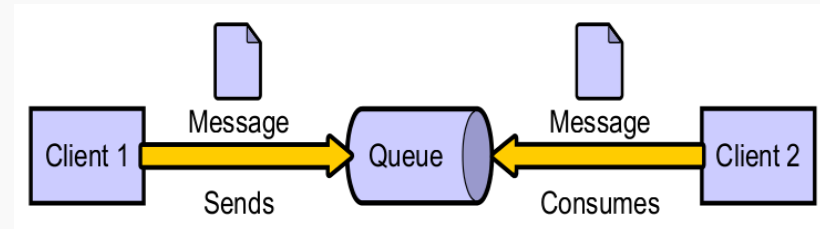
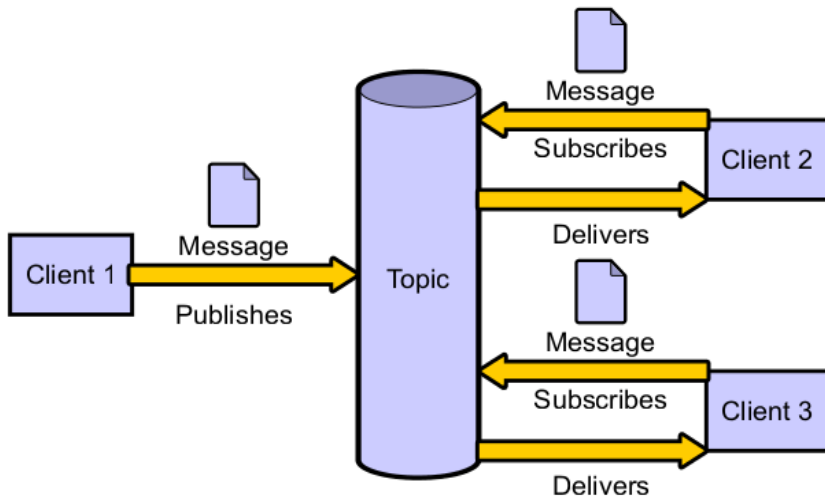
```
@Stateful
@TransactionManagement(BEAN)
public class MySessionBean implements MySession {
    @Resource UserTransaction ut;
    public void method() {
        try {
            ut.begin(); // Открываем транзакцию
            //... какие-то действия
            ut.commit(); // Закрываем транзакцию
        } catch (Exception e) {
            ut.rollback(); // Ошибка – откат транзакции
        }
    }
}
```

EJB Lite & Full

Feature	EJB Lite	EJB		Feature	EJB Lite	EJB
Stateless beans	+	+		Asynchronous invocation		+
Stateful beans	+	+		Interceptors	+	+
Singleton beans	+	+		Declarative security	+	+
Message driven beans		+		Declarative transactions	+	+
No interfaces	+	+		Programmatic transactions	+	+
Local interfaces	+	+		Timer service		+
Remote interfaces		+		EJB 2.X support		+
Web service interfaces		+		CORBA interoperability		+

Java Message Service (JMS)

- Позволяет организовать асинхронный обмен сообщениями между компонентами.
- Две модели доставки сообщений — «подписка» (topic) и «очередь» (queue):



Message-Driven Beans (MDB)

MDB — это компоненты, логика которых является реакцией на события, происходящие в системе.

Особенности MDB:

- Реализуют интерфейс слушателя соответствующего типа сообщений (например, JMS).
- Нет возможности вызова методов «напрямую».
- Нет локальных и удалённых интерфейсов.
- Контейнер может создавать пулы MDB.
- При развёртывании MDB нужно регистрировать в качестве получателей сообщений.

Пример MDB

```
import javax.ejb.*;
import javax.jms.*;

@MessageDriven(mappedName = "jms/MyQueue",
    activationConfig = {
        @ActivationConfigProperty(propertyName =
            "acknowledgeMode", propertyValue =
            "Auto-acknowledge"),
        @ActivationConfigProperty(propertyName =
            "destinationType", propertyValue =
            "javax.jms.Queue")
    })
public class SampleMDB implements MessageListener {
    public void onMessage(Message message) {
        // Do something
    }
}
```

- Позволяют организовать взаимодействие между сетевыми ресурсами по стандартизированному протоколу.
- Ресурсы могут работать на любой платформе.
- Данные «упаковываются» в XML и передаются по HTTP.
- Основные стандарты — SOAP, WSDL и WS-I.
- На платформе Java EE реализуются JAX-WS API и JAX-RS API.
- Основа SOA (Service Oriented Architecture).

Пример реализации веб-сервиса (JAX-WS)

```
package example;

import javax.jws.*;

@WebService
public class SayHello {

    @WebMethod
    public String getGreeting(String name){
        return "Hello " + name;
    }

}
```

Пример реализации клиента веб-сервиса (JAX-WS)

```
import javax.xml.ws.WebServiceRef;

public class WSTest {

    public WSTest() { }

    public static void main(String[] args) {
        SayHelloService service = new SayHelloService();
        SayHello port = service.getSayHelloPort();
        System.out.println(port.sayHello("Duke"));
    }
}
```